Key-Updatable Identity-Based Signature Schemes

Tobias Guggemos[1,2] and Farzin Renan[3]

[1] Ludwig-Maximilians-Universität München, Munich, Germany
[2] German Aerospace Center (DLR), Wessling, Germany
[3] Middle East Technical University, Ankara, Turkey
guggemos@nm.ifi.lmu.de, tobias.guggemos@dlr.de, farzin.renan@gmail.com,

Abstract. Identity-based signature (IBS) schemes eliminate the need for certificate management, thereby reducing communication and computational overhead. A major challenge, however, is the efficient update or revocation of compromised keys, as existing approaches such as revocation lists or periodic key renewal incur significant network costs in dynamic settings. We address this challenge by introducing a symmetric element that enables key updates in IBS schemes through a single multicast message. Our approach achieves logarithmic network overhead in the number of keys, with constant computation and memory costs. We further propose a general framework that transforms any IBS scheme into a key-updatable IBS scheme (KUSS), and formalize the associated security requirements, including token security, forward security, and post-compromise security. The versatility of our framework is demonstrated through five instantiations based on Schnorr-type, pairing-based, and isogeny-based IBS, and we provide a detailed security analysis.

Keywords: Identity-Based Signatures · Key Revocation · Group Communication · ECC · Pairing-based · Isogeny-based Cryptography

1 Introduction

Efficient communication is critical for resource-constrained devices, including IoT sensors, smart lighting, and vehicular networks. Minimizing network overhead directly extends device lifetime and reduces maintenance costs. Traditional certificate-based authentication is ill-suited for such environments: each signature carries certificates, and verification requires scanning Certificate Revocation Lists (CRLs) maintained by Certificate Authorities (CAs). In large-scale dynamic systems, such as Car-to-Car (Car2Car) communication, this results in complex multi-CA trust infrastructures [1].

Identity-based signatures (IBS) address these challenges by binding public keys to user identities. A Trusted Third Party (TTP) generates user keys and publishes a master public key (mpk), allowing verification using only the sender's identity and mpk. This design makes IBS an attractive alternative to Public Key Infrastructure (PKI) in closed or dynamic groups. However, key revocation and

updates remain challenging. Boneh and Franklin's generic key renewal method [2] requires secure channels and incurs network overhead proportional to private key sizes. Similarly, revocation in IBS can lead to high costs, including linear communication overhead [2], expensive pairing or lattice operations [3, 4], or logarithmic growth in key and signature size [5].

Principles of Key Updates. In both PKI and IBS, once keys are distributed, they cannot simply be retracted. PKIs rely on CRLs to mark revoked certificates. In IBS, tracking invalid identities through lists would undermine the goal of avoiding third-party checks [6]. A natural alternative is re-keying: the TTP revokes the current mpk and issues fresh keys to valid users.

1.1 Related Work and Our Contribution

The generic approach for updating of IBS keys, proposed by Boneh and Franklin [2] is too costly for constrained networks, motivating several alternatives.

Hierarchical Keys. Hierarchical Identity-Based Signatures (HIBS) [5] provide flexible role-based structures, promising for sensor networks, but signature and secret key sizes grow with hierarchy depth. Optimizations achieving constant signature sizes [7] require large public keys and still do not support efficient subtree renewal. Tree-based revocation, explored initially for IBE [3] and later adapted to IBS [8], requires multiple pairing operations, making these approaches impractical for constrained settings.

Key Insulation and Updatable Encryption. Key insulation [9] and ciphertext updates [10] inspired our approach. In [9], both master and user keys are updated via tokens, with signatures embedding update information. Our method differs by introducing a symmetric update token that simultaneously updates all users' keys via a single broadcast. Security relies solely on the confidentiality of this token, supporting forward and post-compromise security with minimal overhead.

Contribution. We extend IBS with two new phases (Next, Update) and introduce a symmetric element δ. A TTP distributes a single confidential update token to all valid users, leveraging group key management for efficiency. We further generalize the transformation of any IBS scheme into a Key-Updatable Signature Scheme (KUSS) and formalize the associated security requirements, including token security (UF-KUSS-CMA), forward security, and post-compromise security.

To illustrate the generality of our approach, we apply our framework to five IBS schemes:

GG: by Galindo et al. [11], based on the Schnorr signature [12].
vBNN: by Cao et al. [13], also based on the Schnorr signature.
Hess: by Hess [14], based on bilinear pairings.
BLMQ: Barreto et al. [15], employed bilinear pairings.
CSIIBS: by Shaw et al. [17], based on CSI-FiSh signature [16].

Existing re-keying approaches exhibit significant trade-offs. Standards such as X.509 rely on revocation lists, keeping signature sizes small but imposing heavy

verification and storage costs. HIBS [5] achieves logarithmic update complexity at the expense of larger signatures and verification keys. Key insulation [9, 18] reduces computational costs but does not eliminate network and memory overhead. Revocable IBE schemes [3] achieve efficient logarithmic revocation but are not effectively adapted to IBS, while pairing-based schemes such as **Hess** [14] and **BLMQ** [15] incur high computational costs.

Table 1. Comparison of our approach (**g**IBS) with other re-keying approaches (n = number of group members)

	$\mathcal{O}_{\lvert\sigma\rvert}$	$\mathcal{O}_S$	$\mathcal{O}_V$	$\mathcal{O}_{N_V}$	$\mathcal{O}_{U_{TTP}}$	$\mathcal{O}_{U_{GM}}$	$\mathcal{O}_{N_U}$	$\mathcal{O}_{\lvert U\rvert}$
ACE [19, 20]	1	1	$\log n$	-	1	1	n	n
X.509 [21]	1	1	n	-	1	1	n	n
OCSP [22]	1	1	1	m	1	-	-	1
vBNN-IBS [13]	1	1	n	-	1	1	n	n
Group Signature [23]	1	$\log n$	1	-	$\log n$	1	$\log n$	$\log n$
Group Signature [24]	1	1	1	-	1	1	n	n^2
IBS [25]	1	1	1	-	1	1	n	1
Inline-HIBS [5]	$\log n$	1	$\log n$	-	1	1	$\log n$	n
Pre-Computed-HIBS [5]	1	1	1	-	$\log n$	n	$\log n$	n
Key-Insulation [18]	1	1	1	-	1	1	n	n
Our Solution	**1**	**1**	**1**	**-**	$\boldsymbol{\log n}$	**1**	**1**	$\boldsymbol{\log n}$

$\mathcal{O}_{\lvert\sigma\rvert}$ = signature size, $\mathcal{O}_S$ = signing cost, $\mathcal{O}_V$ = verification cost, $\mathcal{O}_{N_V}$ = network overhead for verifying m signatures, $\mathcal{O}_{U_{TTP}}$ = preparation cost at TTP, $\mathcal{O}_{U_{GM}}$ = processing cost at GM, $\mathcal{O}_{N_U}$ = network overhead for a key update, $\mathcal{O}_{\lvert U\rvert}$ = size of the update message.

Our solution achieves constant-size signatures, lightweight signing and verification, and key update overhead that grows logarithmically with the number of users. Unlike pairing-based methods, it is broadly applicable across signature constructions, making it suitable for resource-constrained and dynamic environments. Furthermore, by leveraging hierarchical update strategies (e.g., the Logical Key Hierarchy (LKH) [26]), key updates can be efficiently transmitted in a single message. A detailed complexity comparison with state-of-the-art approaches is provided in Table 1.

Notation. The security parameter is denoted by λ. For a set $\mathbb{S}$, the notation $s \xleftarrow{\$} \mathbb{S}$ represents sampling an element s uniformly at random from $\mathbb{S}$. A probabilistic polynomial-time (PPT) algorithm A producing output x on input y is denoted as $x \xleftarrow{\$} A(y)$. For a deterministic polynomial-time (DPT) algorithm, this is represented as $x := A(y)$, or $x \leftarrow A(y)$. A function $\mathsf{negl} : \mathbb{N} \to \mathbb{R}$ is called *negligible* if, for every $k \in \mathbb{N}$, there exists an $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$, the inequality $\mathsf{negl}(n) \leq \frac{1}{n^k}$ holds. The concatenation of two strings s_1 and s_2

is denoted by $s_1 \| s_2$. For a positive integer n, $\mathbb{Z}_n := \mathbb{Z}/n\mathbb{Z}$. We use the notation $\forall i \in [T]$ to denote "for all $i \in \{1, 2, \ldots, T\}$".

2 Preliminaries

2.1 Identity-based Signature Scheme

Formally, an IBS scheme consists of three entities: the Key Generation Center (KGC), the signer, and the verifier. The formal definition of an IBS scheme, along with its corresponding security properties, is provided following the syntax in [17].

Definition 2.1 (Identity-based Signature Scheme). *An identity-based signature (IBS) scheme is a quadruple* $(\mathsf{Setup}, \mathsf{Ext}, \mathsf{Sign}, \mathsf{Ver})$ *consisting of four polynomial-time algorithms with the following syntax:*

- $\mathsf{Setup}(1^\lambda) \xrightarrow{\$} (\mathsf{pp}, \mathsf{msk}, \mathsf{mpk})$: *A* PPT *algorithm executed by the KGC, takes the security parameter* λ *as input, outputs the public parameters* pp*, the master secret key* msk*, and the corresponding master public key* mpk*.*
- $\mathsf{Ext}(\mathsf{msk}, \mathsf{id}) \xrightarrow{\$} \mathsf{usk}$: *A* PPT *algorithm executed by the KGC, takes the master secret key* msk *and a member's identity* id*, generates a user's secret key* usk *associated with* id*.*
- $\mathsf{Sign}(\mathsf{usk}, m) \xrightarrow{\$} \sigma$: *A* PPT *algorithm that takes the user's secret key* usk *and a message* m *as input, outputs a signature* σ *for the message* m*.*
- $\mathsf{Ver}(\mathsf{id}, m, \sigma) \rightarrow 0/1$: *A* DPT *algorithm that takes a signer's identity* id*, a message* m*, and a signature* σ*, checks the validity of the signature* σ *on the message* m*.*

Definition 2.2 (Correctness). *An IBS scheme satisfies correctness if for any* $\lambda \in \mathbb{N}$*, and* $\mathsf{id} \in \mathsf{ID}$*, and any message* $m \in \{0,1\}^*$*, the following holds:*

$$Pr\left[\mathsf{Ver}\big(\mathsf{id}, m, \mathsf{Sign}(\mathsf{usk}, m)\big) = 1 \;\middle|\; \begin{array}{c} (\mathsf{pp}, \mathsf{msk}, \mathsf{mpk}) \xleftarrow{\$} \mathsf{Setup}(1^\lambda) \\ \mathsf{usk} \xleftarrow{\$} \mathsf{Ext}(\mathsf{msk}, \mathsf{id}) \end{array} \right] = 1.$$

Definition 2.3 (UF-IBS-CMA Security). *An IBS scheme satisfies unforgeability under chosen identity and chosen message attacks* (UF-IBS-CMA) *if for all* PPT *adversary* $\mathcal{A}$*, there exists a negligible function* negl *such that*

$$\mathsf{Pr}\left[\mathsf{Exp}^{\mathsf{UF\text{-}IBS\text{-}CMA}}_{\mathsf{IBS},\mathcal{A}}(\lambda) = 1\right] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{Exp}^{\mathsf{UF\text{-}IBS\text{-}CMA}}_{\mathsf{IBS},\mathcal{A}}$ *is defined as shown in Fig. 1.*

Input: The challenger $\mathcal{C}$ takes the security parameter λ as input, produces $(\mathsf{pp}, \mathsf{msk}, \mathsf{mpk}) \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$. It gives the $\mathsf{pp}, \mathsf{mpk}$ to the adversary $\mathcal{A}$ while keeping msk secret to itself.

Query Phase: In this phase, $\mathcal{C}$ responds to polynomially many adaptive queries made by $\mathcal{A}$ by following the steps described below:

- **Oracle** $\mathcal{O}_{\mathcal{E}}(\mathsf{msk}, \cdot)$: Upon receiving queries on a user identity id from $\mathcal{A}$, $\mathcal{C}$ outputs the user's secret key $\mathsf{usk} \xleftarrow{\$} \mathsf{Ext}(\mathsf{msk}, \mathsf{id})$ for the given identity id.
- **Oracle** $\mathcal{O}_{\mathcal{S}}(\mathsf{usk}, \cdot)$: Upon receiving queries on a message m and a user identity id from $\mathcal{A}$, $\mathcal{C}$ returns a signature $\sigma \xleftarrow{\$} \mathsf{Sign}(\mathsf{usk}, m)$, where $\mathsf{usk} \xleftarrow{\$} \mathsf{Ext}(\mathsf{msk}, \mathsf{id})$ is the user's secret key associated to the id.

Forgery: In this phase, $\mathcal{A}$ eventually submits a message m^*, user identity id^*, and a forgery σ^*. $\mathcal{A}$ wins the game if $1 \leftarrow \mathsf{Ver}(\mathsf{id}^*, m^*, \sigma^*)$, with the restriction that id^* has not been queried to $\mathcal{O}_{\mathcal{E}}(\mathsf{msk}, \cdot)$ and (m^*, id^*) has not been queried to $\mathcal{O}_{\mathcal{S}}(\mathsf{usk}, \cdot)$.

Fig. 1. $\mathsf{Exp}^{\mathsf{UF\text{-}IBS\text{-}CMA}}_{\mathsf{IBS},\mathcal{A}}$ Experiment.

2.2 Elliptic curves and isogenies

The primary comprehensive reference widely regarded for elliptic curves is Silverman [27]. Let $k := \mathbb{F}_q$ be a finite field where $q := p^n$ for some prime $p > 3$, and positive integer n. An *elliptic curve* E over a field k is a smooth, projective variety of genus 1, defined over k, with a distinguished rational point $\mathcal{O}_E$. For a positive integer ℓ, the *ℓ-tosion subgroup* of an elliptic curve E is defined as $E[\ell] := \{P \in E(\bar{k}) \,|\, [\ell]P = \mathcal{O}_E\}$. An elliptic curve E is called *supersingular* if it has no nontrivial p-torsion points over $\overline{\mathbb{F}}_p$, i.e., $E[p] = \{\mathcal{O}_E\}$. Specifically, if $E/\mathbb{F}_p$ is supersingular, then $\#E(\mathbb{F}_p) = p + 1$.

An *isogeny* is a surjective morphism between elliptic curves that is also a homomorphism with respect to the natural group structure on these curves. Two elliptic curves E_1 and E_2 are said to be *isogenous* over $\mathbb{F}_q$ if there exists an isogeny between them over $\mathbb{F}_q$. Any subgroup $G \subset E(\mathbb{F}_q)$ determines a unique (separable) isogeny $\phi : E \to E' := E/G$ with $\ker(\phi) = G$, up to $\mathbb{F}_q$-isomorphism.

An *endomorphism* is an isogeny from an elliptic curve E to itself. Examples include the *multiplication-by-m* map $[m] : P \mapsto [m]P$ for $m \in \mathbb{Z}$, and the *Frobenius* map $\pi : (x, y) \mapsto (x^q, y^q)$ for $E/\mathbb{F}_q$. The set of all endomorphisms on E, denoted by $\mathrm{End}(E)$, forms a ring under addition and composition, known as the *endomorphism ring* of E. For supersingular elliptic curves, $\mathrm{End}(E)$ is isomorphic to an order in a quaternion algebra, whereas the ring of $\mathbb{F}_p$-endomorphism on E, denoted by $\mathrm{End}_{\mathbb{F}_p}(E)$, is isomorphic only to an order in the imaginary quadratic field $\mathbb{Q}(\sqrt{-p})$. Thus, for a supersingular elliptic curve E over $\mathbb{F}_p$, there is a strict inclusion, such that $\mathrm{End}_{\mathbb{F}_p}(E) \subset \mathrm{End}(E)$.

The ideal class group of an order $\mathfrak{O} \subset \mathbb{Q}(\sqrt{-p})$ is defined as the quotient of the group of invertible fractional ideals $\mathcal{I}_{\mathfrak{O}}$ by the subgroup of principal fractional ideals $\mathcal{P}_{\mathfrak{O}}$, i.e., $\mathrm{Cl}(\mathfrak{O}) := \mathcal{I}_{\mathfrak{O}}/\mathcal{P}_{\mathfrak{O}}$. There is a natural action of the class group $\mathrm{Cl}(\mathfrak{O})$ on the set $\mathcal{E}ll_p(\mathfrak{O})$ of $\mathbb{F}_p$-isomorphism classes of elliptic curves defined over $\mathbb{F}_p$ with endomorphism rings isomorphic to $\mathfrak{O}$. Specifically, for a given $\mathfrak{O}$-ideal $\mathfrak{a}$, let the subgroup $S_{\mathfrak{a}}$ be defined by the intersection of the kernels of the

endomorphisms in $\mathfrak{a}$, i.e., $S_\mathfrak{a} := \bigcap_{\alpha \in \mathfrak{a}} \ker(\alpha)$. As $S_\mathfrak{a}$ is a subgroup of E, we can quotient E by $S_\mathfrak{a}$ and denote the isogenous curve by $\mathfrak{a} * E := E/S_\mathfrak{a}$. The isogeny $\phi_\mathfrak{a} : E \to E/S_\mathfrak{a}$ is well-defiend and unique up to $\mathbb{F}_p$-isomorphism. The class group $\mathrm{Cl}(\mathfrak{O})$ acts via the operator $*$ on the set $\mathcal{E}ll_p(\mathfrak{O})$, i.e., $* : \mathrm{Cl}(\mathfrak{O}) \times \mathcal{E}ll_p(\mathfrak{O}) \to \mathcal{E}ll_p(\mathfrak{O})$ freely and transitively.

As in [16], an embedding $\mathbb{Z}/N\mathbb{Z} \hookrightarrow \mathrm{Cl}(\mathfrak{O})$ is defined by $a \mapsto \mathfrak{g}^a$, where $\mathrm{Cl}(\mathfrak{O})$ is cyclic with generator $\mathfrak{g}$. Elements of $\mathrm{Cl}(\mathfrak{O})$ are expressed as $\mathfrak{a} = \mathfrak{g}^a$, with a sampled randomly from $\mathbb{Z}/N\mathbb{Z}$ such that $N = \#\mathrm{Cl}(\mathfrak{O})$. Using the shorthand $[a]$ for $\mathfrak{g}^a$ and $[a]E$ for $\mathfrak{g}^a * E$, it follows that $[a][b]E = [a+b]E$.

2.3 Computational Assumptions

Our analysis relies on the following foundational group properties and computational hardness assumptions. A key property underlying our reasoning is the *Latin square property*, which ensures that the operation within a group produces uniformly distributed elements:

Lemma 2.1 (Latin square property). *Let $(G, \circ)$ be a group of order $|G| = k$, and let $a \in G$. Then, for any $b, c \in G$ chosen uniformly at random,*

$$P(c = a \circ b) = \frac{1}{k}.$$

As a corollary, the operation of two randomly drawn elements from G results in a uniformly distributed random element of G. Specifically, in the group $(\mathbb{Z}_p^*, \cdot)$:

Lemma 2.2 (Randomness preservation in $(\mathbb{Z}_p^*, \cdot)$). *For any $a \in \mathbb{Z}_p^*$ and a random $b \in \mathbb{Z}_p^*$, the product $c = a \cdot b$ is uniformly distributed in $\mathbb{Z}_p^*$.*

Proof. Let p be a prime and $(\mathbb{Z}_p^*, \cdot)$ a group of order $p-1$. For any $x \in \mathbb{Z}_p^*$,

$$P(x = x' \mid x' \xleftarrow{\$} \mathbb{Z}_p^*) = \frac{1}{p-1} = P(x = a \cdot b),$$

showing that the product of a random b with a yields a random group element $c \in \mathbb{Z}_p^*$.

Moreover, the primary computational assumption underlying our isogeny-based instantiation, introduced in Section 4.2, is the *Multi-Target Group Action Inverse Problem (MT-GAIP)*, which captures the difficulty of inverting group actions derived from isogenies. The CSI-FiSh signature scheme [16], and consequently **CSIIBS** [17], relies on the hardness of random instances of MT-GAIP.

*Problem 2.1 (**Multi-Target Group Action Inverse Problem (MT-GAIP) [16]**).* Suppose $E_0, E_1, \ldots, E_k$ are $k+1$ supersingular elliptic curves defined over $\mathbb{F}_p$, with $\mathrm{End}_{\mathbb{F}_p}(E_i) \cong \mathfrak{O}$ for all $0 \le i \le k$. Find an ideal $\mathfrak{a} \subset \mathfrak{O}$ such that $E_i = \mathfrak{a} * E_j$ for some distinct indices $i, j \in \{0, \ldots, k\}$.

3 Key-Updatable Identity-Based Signature Scheme

This section introduces the Key-Updatable Identity-Based Signature Scheme (KUSS), formalizes its framework and security properties, and demonstrates its use in designing a protocol for IBS key updates.

Symmetric Group Re-keying. In a communication group, each participant, called a Group Member (GM), holds a copy of the same symmetric group key, which is used to ensure confidentiality and authenticate group communications. Updating one member's key therefore requires updating all members' keys; this process is known as re-keying. Efficient symmetric re-keying schemes typically preserve three security properties: **i.** Forward Access Control, **ii.** Backward Access Control, and **iii.** Key Independence [28].

Although decentralized and distributed re-keying mechanisms exist [29], we focus on centralized approaches, motivated by their structural similarity to IBS key management. In such approaches, a Group Controller Key Server (GCKS) manages group membership and securely distributes key updates. Protocols such as G-IKEv2 [30], COSE [19], and key management mechanisms like the LKH [26] enable efficient key updates.

Our proposed IBS key update mechanism leverages these efficient symmetric re-keying techniques. In the remainder of this paper, we assume symmetric key updates are efficient and use LKH as an example of a centralized binary-tree approach, where update complexity scales as $\mathcal{O}(\log n)$.

Basic Idea. At the heart of IBS is the mapping of a user's identity string to the corresponding user secret key usk via a mathematical relation with the master secret key. Building on this principle, our approach—illustrated in Fig. 2—is motivated by the following observation:

Update token $\boldsymbol{\delta}$. In many schemes, when the master secret key is renewed, the transition from msk_0 to msk_1 can be captured by a mathematical object δ, even if the keys are chosen independently. Since msk is used to derive both the master public key mpk and user secret key usk, updates to these keys can also be expressed in terms of δ (or a related element). Crucially, as long as msk_0 remains uncompromised, knowledge of δ reveals nothing about either msk_0 or msk_1.

In the context of user key updates—or more generally, key revocation—it is both secure and efficient for the TTP to distribute δ to all valid users rather than issuing entirely new keys individually. Each user can then independently update their keys to usk_1 and mpk_1. In case of key compromise, it suffices to prevent the update from reaching users outside the system. This approach also supports key revocation by distributing the update to all parties except the revoked user.

Update accumulator $\boldsymbol{\Delta}$. When keys are updated multiple times, the cumulative difference from the original key material can still be expressed through the mathematical object δ. In all schemes considered here, the overall change is represented as the accumulation of individual updates, denoted by Δ.

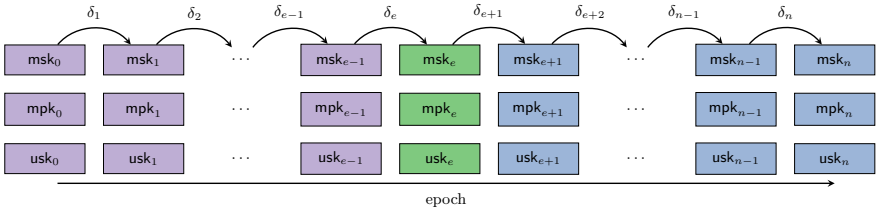

Fig. 2. Idea of updating IBS keys with an update token δ over epochs e

Example. Consider the **Hess** IBS scheme [14]. Let msk_1 be the current master secret key, and let the TTP select a fresh key msk_2 for a re-key operation. Since both msk_1 and msk_2 belong to a cyclic group $\mathbb{Z}_p^*$, there exists $\delta_1 \in \mathbb{Z}_p^*$ such that $\mathsf{msk}_2 = \delta_1 \cdot \mathsf{msk}_1$. In **Hess** IBS, the master public key is $\mathsf{mpk} = \mathsf{msk} \circ P$, and the user secret key is $\mathsf{usk} = \mathsf{msk} \circ \mathsf{H}(\mathsf{id})$, where $\mathsf{H}(\mathsf{id}) \in \langle P \rangle$ is a hash of the user's identity. Hence, transitioning from msk_1 to msk_2 updates both mpk and usk by δ_1:

	msk	mpk	usk
epoch 1	msk_1	$\mathsf{msk}_1 \circ P$	$\mathsf{msk}_1 \circ \mathsf{H}(\mathsf{id})$
epoch 2	$\delta_1 \cdot \mathsf{msk}_1$	$\delta_1 \cdot \mathsf{msk}_1 \circ P$	$\delta_1 \cdot \mathsf{msk}_1 \circ \mathsf{H}(\mathsf{id})$

After a subsequent update $\delta_2 \xleftarrow{\$} \mathbb{Z}_p^*$, the master secret becomes $\mathsf{msk}_3 = \delta_2 \cdot \mathsf{msk}_2 = \Delta_2 \cdot \mathsf{msk}_1$, where $\Delta_2 = \delta_2 \cdot \delta_1$. The **Hess** scheme allows seamless key updates because δ can be incorporated directly into user keys, without modifying signing or verification algorithms. Once the updated keys are computed, δ can be discarded. In contrast, other key-updatable schemes may require storing δ (or Δ) for signing or verification.

Formal Definition. The information each entity needs to update their key material, denoted δ, is called the *update token*. A *key update* refers to distributing this token and computing the corresponding new keys. The term *epoch* denotes the interval between successive updates. The update accumulator Δ, also called the *group shared secret*, is accessible only to non-revoked users during a given epoch and is not part of the public parameters. An IBS scheme in which the mathematical relation between the master and user keys allows key updates is called a *Key-Updatable Identity-Based Signature Scheme* (KUSS), formally defined as follows:

Definition 3.1 (Key-Updatable Identity-Based Signature). *A Key-Updatable Identity-Based Signature Scheme (KUSS) for message space $\mathcal{M}$ is a tuple of polynomial-time algorithms*

$$\mathsf{KUSS} := (\mathsf{Setup}, \mathsf{Ext}, \mathsf{Next}, \mathsf{Update}, \mathsf{Sign}, \mathsf{Ver}),$$

where $(\mathsf{Setup}, \mathsf{Ext}, \mathsf{Sign}, \mathsf{Ver})$ *are inherited from a standard IBS scheme (Definition 2.1), and* $(\mathsf{Next}, \mathsf{Update})$ *are additional algorithms defined as follows:*

$\mathsf{Next}(\lambda) \xrightarrow{\$} \delta$**:** *A PPT algorithm executed by the KGC that takes the security parameter λ and outputs the update token δ for the next epoch.*

$\mathsf{Update}_{\delta_{e+1}}(k_e) \to k_{e+1}$**:** *A DPT algorithm that takes the key material $k_e := (\mathsf{msk}_e, \mathsf{mpk}_e, \mathsf{usk}_e, \Delta_e)$ from epoch e and the update token δ_{e+1}, and outputs the updated key material k_{e+1} for epoch $e+1$.*

Definition 3.2 (Correctness). *A Key-Updatable Signature Scheme (KUSS), satisfies correctness if for any $\lambda \in \mathbb{N}$, any epoch e, and $\mathsf{id} \in \mathsf{ID}$, and any message $m \in \mathcal{M}$, the following holds:*

$$\Pr\left[\mathsf{Ver}_{\mathsf{mpk}_e}\big(\mathsf{id}, m, \mathsf{Sign}(\mathsf{usk}_e, m)\big) = 1 \;\middle|\; \begin{array}{c}(\mathsf{pp}, \mathsf{msk}_0, \mathsf{mpk}_0) \xleftarrow{\$} \mathsf{Setup}(1^\lambda),\\ \mathsf{usk}_0 \xleftarrow{\$} \mathsf{Ext}(\mathsf{msk}_0, \mathsf{id}),\\ \delta_i \xleftarrow{\$} \mathsf{Next}(\lambda),\ 1 \le i \le e,\\ k_i \leftarrow \mathsf{Update}_{\delta_i}(k_{i-1}),\ \textit{where}\\ k_i := (\mathsf{msk}_i, \mathsf{mpk}_i, \mathsf{usk}_i, \Delta_i), 1 \le i \le e.\end{array}\right] = 1.$$

Transformability Conditions. The KUSS is generally applicable to all IBS schemes that satisfy the following conditions:

Property 3.1 (Symmetric Integration). It is possible to select a shared secret Δ such that it can be integrated into the signing process with usk, i.e., in $\mathsf{Sign}(\mathsf{usk}, m)$, without increasing the number of signatures produced.

Concretely, since the key update process is identical for all users, the update token δ and the accumulator Δ can be interpreted as symmetric keys shared among the users and the TTP. Secretly updating this symmetric element implicitly updates the corresponding asymmetric keys, because signatures produced with an invalid symmetric element become invalid. In other words, any IBS scheme can be generically transformed into a KUSS by incorporating such a symmetric element into the signing process. For example, if Δ is used as a symmetric key in $\mathsf{HMAC}(\Delta, m)$ to "pre-sign" the message, then verification satisfies

$$\mathsf{Ver}(\mathsf{Sign}(\mathsf{usk}, m)) = 1 \;\Leftrightarrow\; \mathsf{Ver}(\mathsf{Sign}(\mathsf{usk}, \mathsf{HMAC}(\Delta, m))) = 1.$$

While Property 3.1 establishes that a shared secret Δ can be integrated symmetrically into the signing process, differences in security and performance may arise in a concrete implementation using a symmetric update token. For example, maintaining a secondary signing key could require additional storage and management, and the signing operation might involve extra computation. Security considerations may also differ between a secondary signing key and an updated one, though in schemes such as the **Hess** scheme [14], these differences are nominal.

If the update tokens (or the shared secret Δ) can be seamlessly incorporated into both the signing and verification keys, we classify these keys as **hybrid**. The following properties define criteria that a concrete KUSS instantiation should satisfy if such integration is possible only for signing or verification.

Property 3.2 (Signing Key Integration). The user secret key usk and the accumulator Δ_e can participate in a binary operation during signing, such that the updated key usk_e signs messages in the same way as the original usk in $\mathsf{Sign}(\mathsf{usk}, m)$.

Property 3.3 (Public Key Integration). The master public key mpk and the accumulator Δ_e can participate in a binary operation during verification, such that the updated key mpk_e verifies signatures in the same way as the original mpk in $\mathsf{Ver}_{\mathsf{mpk}}(\mathsf{id}, m, \sigma)$.

3.1 Security Model

Having introduced key updates in IBS schemes via an update token, we now formalize the security considerations. This construction can be seen as a hybrid of a standard PKI and a group key management system. Security definitions for both paradigms have been extensively studied and standardized (e.g., [31, 32, 33, 34]). For IBS schemes, the fundamental security notion is UF-IBS-CMA (Definition 2.3). We emphasize that the introduction of the shared secret Δ and the corresponding update mechanism does not weaken the security guarantees of the constructions described in Section 4. In the following, we examine security properties inspired by Updatable Encryption [10], adapted to the context of Key-Updatable Identity-Based Signature Schemes (KUSS).

Token Security. Integrating a shared secret into a signature scheme does not weaken the security of the underlying construction, as it remains protected under the UF-IBS-CMA definition. Building on this foundation, we formally define the token security property for KUSS as follows.

Definition 3.3 (UF-KUSS-CMA Security). *A KUSS scheme is secure against unforgeability under chosen-message-identity-and-epoch attacks (UF-KUSS-CMA) if for all PPT adversaries $\mathcal{A}$, there exists a negligible function* negl *such that*

$$\Pr\left[\mathsf{Exp}^{\textit{UF-KUSS-CMA}}_{\textit{KUSS},\mathcal{A}}(\lambda) = 1\right] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{Exp}^{\textit{UF-KUSS-CMA}}_{\textit{KUSS},\mathcal{A}}$ is defined as illustrated in Fig. 3.

Forward and Post-Compromise Security. These properties capture the resilience of a KUSS against key compromises, ensuring that both past and future signatures remain secure even if a secret key is exposed in a given epoch.

***(i)* Forward Security** An adversary compromising any secret key (msk, usk, or Δ) in some epoch e^* does not gain any advantage in forging signatures for previous epochs $e < e^*$. That is, all signatures and signing keys from epochs *before* the compromise retain their integrity.

***(ii)* Post-Compromise Security** An adversary compromising any secret key (msk, usk, or Δ) in some epoch e^* does not gain any advantage in forging signatures for subsequent epochs $e > e^*$. That is, all signatures and keys from epochs *after* the compromise remain secure.

Input: The challenger $\mathcal{C}$, given the security parameter λ, runs $(\mathsf{pp}, \mathsf{msk}_0, \mathsf{mpk}_0) \leftarrow \mathsf{Setup}(1^\lambda)$. It provides pp and mpk_0 to the adversary $\mathcal{A}$ while keeping msk_0 secret. Finally, it initializes an empty set $\mathsf{idList} = \emptyset$.
Query Phase: In this phase, $\mathcal{C}$ responds to polynomially many adaptive queries made by $\mathcal{A}$ by following the steps described below:

- **Oracle** $\mathcal{O}_{\mathsf{Ext}}(\mathsf{msk}_e, \cdot)$: Upon receiving a query on a user identity id from $\mathcal{A}$ at epoch e, if $\mathsf{id} \notin \mathsf{idList}$, the challenger $\mathcal{C}$ generates the corresponding user secret key $\mathsf{usk}_e \xleftarrow{\$} \mathsf{Ext}(\mathsf{msk}_e, \mathsf{id})$ and updates the list as $\mathsf{idList} \leftarrow \mathsf{idList} \cup \{\mathsf{id}\}$. Otherwise, $\mathcal{C}$ aborts.
- **Oracle** $\mathcal{O}_{\mathsf{N}}(\lambda)$: On receiving queries at epoch e from $\mathcal{A}$, $\mathcal{C}$ generates a new update token for epoch $e+1$ via $\delta_{e+1} \xleftarrow{\$} \mathsf{Next}(\lambda)$.
- **Oracle** $\mathcal{O}_{\mathsf{U}}(\cdot, \cdot)$: On receiving queries from $\mathcal{A}$ to update to a new epoch, $\mathcal{C}$ uses the update token δ_{e+1} along with the current key materials $k_e := (\mathsf{msk}_e, \mathsf{mpk}_e, \mathsf{usk}_e, \Delta_e)$, (for all $\mathsf{id} \in \mathsf{idList}$), for epoch e to compute the updated key materials $k_{e+1} \leftarrow \mathsf{Update}_{\delta_{e+1}}(k_e)$, where $k_{e+1} := (\mathsf{msk}_{e+1}, \mathsf{mpk}_{e+1}, \mathsf{usk}_{e+1}, \Delta_{e+1})$. It then returns mpk_{e+1} to $\mathcal{A}$.
- **Oracle** $\mathcal{O}_{\mathsf{Sign}}(\mathsf{usk}_e, \cdot)$: On receiving queries on a message m and a user identity id at epoch e from $\mathcal{A}$, $\mathcal{C}$ responds with a signature $\sigma \xleftarrow{\$} \mathsf{Sig}(m, \mathsf{usk}_e)$, where $\mathsf{usk}_e \xleftarrow{\$} \mathsf{Ext}(\mathsf{msk}_e, \mathsf{id})$ or $\mathsf{usk}_e \leftarrow \mathsf{Update}_{\delta_e}(\cdot, \cdot, \mathsf{usk}_{e-1}, \cdot)$ is the user secret key associated to the identity id at epoch e.

Forgery: In this phase, $\mathcal{A}$ eventually submits a message m^*, a user identity id^*, and a forgery σ^*, along with e^*. $\mathcal{A}$ wins the game if $1 \leftarrow \mathsf{Ver}_{\mathsf{mpk}_{e^*}}(\mathsf{id}^*, m^*, \sigma^*)$, with the restriction that id^* and (m^*, id^*) has not been queried to $\mathcal{O}_{\mathsf{Ext}}$ and $\mathcal{O}_{\mathsf{Sign}}$, respectively, and $\mathcal{O}_{\mathsf{N}}$ has not queried at epoch e^*.

Fig. 3. $\mathsf{Exp}^{\mathsf{UF\text{-}KUSS\text{-}CMA}}_{\mathsf{KUSS},\mathcal{A}}$ Experiment.

3.2 gIBS: A protocol for IBS key distribution and updates

RFC 4046 [28] defines a symmetric group re-keying architecture with the following relevant keys:

Key Encryption Key (KEK): A shared secret between a Group Member (GM) and the Group Controller Key Server (GCKS), e.g., established via an authenticated Diffie-Hellman exchange.
Group Key Encryption Key (GKEK): A secret shared among all group members, used solely to secure other keys. It may be part of an efficient re-keying mechanism such as LKH.

This architecture enables seamless integration of IBS and KUSS keys. User secrets (usk) and key updates (δ, Δ) can thus be integrated and securely communicated to new and existing GMs via RFC 4046 [28]:

Setup: The GCKS initializes the group by setting $(\mathsf{pp}, \mathsf{msk}_0, \mathsf{mpk}_0, \Delta_0) \xleftarrow{\$} \mathsf{Setup}(\lambda)$.
Join: A client with identity id wishes to join the communication group.

1. The GCKS generates $\delta_{e+1} \xleftarrow{\$} \mathsf{Next}(\lambda)$ to move to the next epoch and sends it to existing clients encrypted under GKEK.
2. All clients of epoch e call $(\mathsf{usk}_{e+1}, \mathsf{mpk}_{e+1}, \Delta_{e+1}) \leftarrow \mathsf{Update}_{\delta_{e+1}}(\mathsf{usk}_e, \mathsf{mpk}_e, \Delta_e)$.
3. The GCKS calls $(\mathsf{msk}_{e+1}, \mathsf{mpk}_{e+1}, \Delta_{e+1}) \leftarrow \mathsf{Update}_{\delta_{e+1}}(\mathsf{msk}_e, \mathsf{mpk}_e, \Delta_e)$.

4. The GCKS calls $\mathsf{usk}_{e+1} \xleftarrow{\$} \mathsf{Ext}(\mathsf{msk}_{e+1}, \mathsf{id})$ and sends $(\mathsf{pp}, \mathsf{usk}_{e+1}, \mathsf{mpk}_{e+1}, \Delta_{e+1})$ to the joining client secured by the corresponding KEK.

Leave: A client with id is excluded from the communication group.

1. The GCKS calls $\delta_{e+1} \xleftarrow{\$} \mathsf{Next}(\lambda)$ in order to move to the next epoch and sends a re-key message including δ_{e+1} to all clients except id secured by a new GKEK, which is efficiently done e.g. by LKH.
2. All clients of epoch e call $(\mathsf{usk}_{e+1}, \mathsf{mpk}_{e+1}, \Delta_{e+1}) \leftarrow \mathsf{Update}_{\delta_{e+1}}(\mathsf{usk}_e, \mathsf{mpk}_e, \Delta_e)$.
3. The GCKS calls $(\mathsf{msk}_{e+1}, \mathsf{mpk}_{e+1}, \Delta_{e+1}) \leftarrow \mathsf{Update}_{\delta_{e+1}}(\mathsf{msk}_e, \mathsf{mpk}_e, \Delta_e)$.

Communication: All clients of epoch e call $\sigma \xleftarrow{\$} \mathsf{Sign}(\mathsf{usk}_e, m)$ and $\mathsf{Ver}_{\mathsf{mpk}_e}(\mathsf{id}, m, \sigma)$ in order to sign and verify messages for epoch e.

4 Instantiations of KUSS

In this section, we present five instantiations of IBS schemes adapted to the KUSS framework. Specifically, Section 4.1 addresses elliptic curve-based IBS schemes, introducing two pairing-based and two Schnorr-type adaptations. Section 4.2, on the other hand, details an instantiation for adapting an isogeny-based IBS scheme to KUSS. Furthermore, we examine the modifications required to ensure compliance with Properties 3.1, 3.2, and 3.3, as established earlier.

4.1 Schnorr-type and Pairing-based IBS Schemes Adaptations

The integration of the shared secret into the pairing-based IBS schemes **Hess** [14] and **BLMQ** [15] is illustrated in Fig. 4, while its incorporation into the Schnorr-type IBS schemes **GG** [11] and **vBNN** [13] is presented in Fig. 5.

These schemes, all of which rely on the (elliptic curve) discrete logarithm problem, transform trivially under Property 3.1. In particular, the update token δ_e for an epoch e is derived by sampling a random element from $\mathbb{Z}_p^*$. The multiplicative accumulator for epoch e is then defined as $\Delta_e := \prod_{j=0}^{e} \delta_j$. Consequently, the master secret key msk_e and master public key mpk_e at an arbitrary epoch e are updated as $\mathsf{msk}_e = \Delta_e \cdot \mathsf{msk}$ and $\mathsf{mpk}_e = (\Delta_e \cdot \mathsf{msk})P$, respectively, where P denotes a generator point on the elliptic curve. By adapting the computation of usk and mpk to align with their roles in the signing and verification algorithms of the schemes, all four schemes can seamlessly incorporate a **group shared secret** without modifying the structure of the signature.

The two pairing-based KUSS schemes, **Hess+g** and **BLMQ+g**, as illustrated in Fig. 4, additionally satisfy Properties 3.2 and 3.3. Specifically, when the updated user secret key usk_e is utilized for signing, the verification algorithm requires only the corresponding updated master public key mpk_e and, in the case of **BLMQ+g**, the updated generator Q_e as well.

In the two Schnorr-type KUSS schemes, **GG+g** and **vBNN+g**, as depicted in Fig. 5, the updated user secret key $\mathsf{usk}_e = (u_e, R)$ inherently includes a secondary Schnorr signature component, u_e. Seamlessly updating usk_e would require recomputing u_e, which is infeasible without access to the system's msk. To

	Hess+g	**BLMQ+g**
Public param.	$\mathsf{H}_1 : \{0,1\}^* \to \mathbb{G}^*, P \xleftarrow{\$} \mathbb{G}^*$ $\mathsf{H}_2 : \{0,1\}^* \to \mathbb{Z}_p^*$ $e : \mathbb{G} \times \mathbb{G} \to \mathbb{Z}_p^*$	$\mathsf{H}_{1,2} : \{0,1\}^* \to \mathbb{Z}_p^*$ $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{Z}_p^*$ $Q \xleftarrow{\$} \mathbb{G}_2, P := \psi(Q) \in \mathbb{G}_1$ $q := e(P, Q)$
Extract	$\mathsf{usk}_e := (\Delta_e \cdot \mathsf{msk})\mathsf{H}_1(\mathsf{id})$	$\mathsf{usk}_e := \frac{1}{(\mathsf{msk}+\mathsf{H}_1(\mathsf{id}))\cdot\Delta_e} P$
Sign	$x \xleftarrow{\$} \mathbb{Z}_p^*, Q \xleftarrow{\$} \mathbb{G}^*$ $r := e(Q, P)^x, h := \mathsf{H}_2(m, r)$ $S_e := h\,\mathsf{usk}_e + xQ$ $\sigma_e := (h, S_e)$	$x \xleftarrow{\$} \mathbb{Z}_p^*, r := q^x$ $h := \mathsf{H}_2(m, r)$ $S_e := (x + h)\mathsf{usk}_e$ $\sigma_e := (h, S_e)$
Verify	$\tilde{r} := e(S_e, P) \cdot e(\mathsf{H}_1(\mathsf{id}), -\mathsf{mpk}_e)^h$ $h \stackrel{?}{=} \mathsf{H}_2(m, \tilde{r})$	$\tilde{r} := e(S_e, \mathsf{H}_1(\mathsf{id})\,Q_e + \mathsf{mpk}_e) \cdot q^{-h}$ $h \stackrel{?}{=} \mathsf{H}_2(m, \tilde{r})$

Fig. 4. Hess and **BLMQ** adaptations.

address this limitation, we propose incorporating Δ_e – a value known exclusively to valid group members – into the verification process through multiplication with the auxiliary value R. However, this approach does not comply with Property 3.3. Alternatively, the signature σ_e could be extended to include $\Delta_e R$. That said, if the verifier is a group member, they can compute $\Delta_e R$ independently, which is significantly more efficient than embedding it into the signature.

	GG+g	**vBNN+g**
Public param.	$\mathsf{H}_{1,2} : \{0,1\}^* \to \mathbb{Z}_p^*$ $P \xleftarrow{\$} \mathbb{G}^*$	$\mathsf{H}_{1,2} : \{0,1\}^* \to \mathbb{Z}_p^*$ $P \xleftarrow{\$} \mathbb{G}^*$
Extract	$r \xleftarrow{\$} \mathbb{Z}_p^*, R := rP$ $u_e := (r + \mathsf{msk} \cdot \mathsf{H}_1(R, \mathsf{id})) \cdot \Delta_e$ $\mathsf{usk}_e := (u_e, R)$	$r \xleftarrow{\$} \mathbb{Z}_p^*, R := rP$ $u_e := (r + \mathsf{msk} \cdot \mathsf{H}_1(R, \mathsf{id})) \cdot \Delta_e$ $\mathsf{usk}_e := (u_e, R)$
Sign	$x \xleftarrow{\$} \mathbb{Z}_p^*, X := xP$ $h := \mathsf{H}_2(\mathsf{id}, m, X)$ $s_e := x + h \cdot u_e$ $\sigma_e := (s_e, R, X)$	$x \xleftarrow{\$} \mathbb{Z}_p^*, X := xP$ $h := \mathsf{H}_2(\mathsf{id}, m, R, X)$ $s_e := x + h \cdot u_e$ $\sigma_e := (s_e, R, h)$
Verify	$c := \mathsf{H}_1(R, \mathsf{id})$ $d := \mathsf{H}_2(\mathsf{id}, m, X)$ $s_e P \stackrel{?}{=} X + d(\Delta_e R + c\,\mathsf{mpk}_e)$	$c := \mathsf{H}_1(R, \mathsf{id})$ $\tilde{X} := s_e P - h(\Delta_e R + c\,\mathsf{mpk}_e)$ $h \stackrel{?}{=} \mathsf{H}_2(\mathsf{id}, m, R, \tilde{X})$

Fig. 5. GG and **vBNN** adaptations.

One can show that the correctness of the above four adaptations directly follows from the correctness of their respective underlying IBS schemes.

4.2 Isogeny-based Adaptation

To construct an isogeny-based KUSS, we build upon the IBS scheme derived from the CSI-FiSh signature developed by Shaw et al. [17].

In the isogeny-based IBS setting, the master secret key and master public key are defined as sequences of S_0 components, namely $\mathsf{msk} := \{s_i\}_{i=1}^{S_0}$ and $\mathsf{mpk} := \{E_i := [s_i]E_0\}_{i=1}^{S_0}$. Consequently, both the token and accumulator are also represented as S_0-element sequences. Unlike the multiplicative accumulator used in the elliptic curve instantiation (Section 4.1), the isogeny-based setting employs an additive accumulator.

For each epoch e, an update token is generated as $\boldsymbol{\delta}_e = \{\delta_{i,e}\}_{i=1}^{S_0}$ with $\delta_{i,e} \xleftarrow{\$} \mathbb{Z}_N$, and the additive accumulator is updated recursively as $\boldsymbol{\Delta}_e = \boldsymbol{\Delta}_{e-1} + \boldsymbol{\delta}_e$, where $\Delta_{i,e} = \sum_{j=0}^{e} \delta_{i,j}$ for $i \in [S_0]$. The master secret and public keys at epoch e are then defined as $\mathsf{msk}_e = \{s_i + \Delta_{i,e}\}_{i=1}^{S_0}$ and $\mathsf{mpk}_e = \{E_i^{(e)}\}_{i=1}^{S_0} := \{[\Delta_{i,e}]E_i\}_{i=1}^{S_0}$. Here, $s_i + \Delta_{i,e}$ incorporates the cumulative contribution of the accumulator into the initial secret, while $E_i^{(e)}$ denotes the corresponding updated public key component.

As described in [17], the public parameters consist of a prime p, base curve E_0, generator $\mathfrak{g}$, class number $h_{\mathfrak{O}} = N$, and integers T_1, T_2, S_0, S_1 with $T_1 < S_0 = 2^{n_0} - 1$ and $T_2 < S_1 = 2^{n_1} - 1$. The hash functions are defined as $\mathsf{H}_1 : \{0,1\}^* \to [0, S_0]^{T_1 S_1}$ and $\mathsf{H}_2 : \{0,1\}^* \to [0, S_1]^{T_1 T_2}$. Based on these parameters, the extraction, signing, and verification procedures are detailed as follows:

Extraction. Given msk_e and an identity id at epoch e, the algorithm samples $r_{i,j} \xleftarrow{\$} \mathbb{Z}_N$ and computes $R_{i,j} = [r_{i,j}]E_0$ for $i \in [T_1]$, $j \in [S_1]$. The hash $\mathbf{u} = \{u_i\}_{i=1}^{T_1 S_1} := \mathsf{H}_1(\{R_{i,j}\}_{i=1,j=1}^{T_1,S_1} \| \mathsf{id})$ is evaluated. Let $s_0 := 0$. For each pair (i, j), compute

$$x_{i,j}^{(e)} := r_{i,j} - s_{u_i} - \Delta_{u_i,e} \pmod{N}.$$

Define $\mathbf{x}^{(e)} := \{x_{i,j}^{(e)}\}_{i=1,j=1}^{T_1,S_1}$, and set $\mathsf{usk}_e := (\mathbf{u}, \mathbf{x}^{(e)})$.

Signing. To sign m at epoch e, let $x_{i,0}^{(e)} := 0$ for $i \in [T_1]$. Compute $X_{i,j} := [x_{i,j}^{(e)}]E_{u_i}^{(e)}$ for $i \in [T_1]$, $j \in [S_1]$. Sample $k_{i,j} \xleftarrow{\$} \mathbb{Z}_N$ and set $K_{i,j} := [k_{i,j}]E_{u_i}^{(e)}$ for $i \in [T_1]$, $j \in [T_2]$. The hash $\mathbf{v} = \{v_{i,j}\}_{i=1,j=1}^{T_1,T_2} := \mathsf{H}_2(\{K_{i,j}\}_{i=1,j=1}^{T_1,T_2} \| m)$ is computed, and

$$z_{i,j} := k_{i,j} - x_{i,v_{i,j}}^{(e)} \pmod{N}.$$

Define $\mathbf{z} := \{z_{i,j}\}_{i=1,j=1}^{T_1,T_2}$ and $\mathbf{X} := \{X_{i,j}\}_{i=1,j=1}^{T_1,S_1}$. The signature is $\sigma = (\mathbf{v}, \mathbf{z}, \mathbf{X})$.

Verification. Given (m, σ, id), recompute $\mathbf{u} = \mathsf{H}_1(\{X_{i,j}\}_{i=1,j=1}^{T_1,S_1} \| \mathsf{id})$. Then, for all (i, j), compute

$$K'_{i,j} := \begin{cases} [z_{i,j}]E_{u_i}^{(e)}, & v_{i,j} = 0, \\ [z_{i,j}]X_{i,v_{i,j}}, & v_{i,j} \neq 0. \end{cases}$$

Finally, compute $\mathbf{v}' := \mathsf{H}_2(\{K'_{i,j}\}_{i=1,j=1}^{T_1,T_2} \| m)$ and accept iff $\mathbf{v} = \mathbf{v}'$.

Correctness. For all $(\mathsf{pp}, \mathsf{msk}_0, \mathsf{mpk}_0) \leftarrow \mathsf{Setup}(1^\lambda)$, any message $m \in \mathcal{M}$, and any signature $\sigma = (\mathbf{v}, \mathbf{z}, \mathbf{X}) \leftarrow \mathsf{Sign}(\mathsf{usk}_e, m)$ with $\mathsf{usk}_e = (\mathbf{u}, \mathbf{x}^{(e)})$ from Ext or Update, it suffices to show that $K'_{i,j} = K_{i,j}$ for all $i \in [T_1]$, $j \in [T_2]$.

First, recover $\mathbf{u}$ via $\mathsf{H}_1(\{X_{i,j}\}_{i=1,j=1}^{T_1,S_1} \| \mathsf{id})$, noting that $X_{i,j} = R_{i,j}$ for $i \in [T_1]$, $j \in [S_1]$. Indeed,

$$X_{i,j} = [x_{i,j}^{(e)}]E_{u_i}^{(e)} = [r_{i,j} - s_{u_i} - \Delta_{u_i,e}][s_{u_i} + \Delta_{u_i,e}]E_0 = [r_{i,j}]E_0 = R_{i,j}.$$

Next, for $\mathbf{v}$, consider

$$K'_{i,j} = \begin{cases} [z_{i,j}]E_{u_i}^{(e)}, & v_{i,j} = 0, \\ [z_{i,j}]X_{i,v_{i,j}}, & v_{i,j} \neq 0, \end{cases} \qquad (*)$$

which evaluates to $[k_{i,j}]E_{u_i}^{(e)}$ in both cases. Hence, $\{K'_{i,j}\}_{i=1,j=1}^{T_1,T_2} = \{K_{i,j}\}_{i=1,j=1}^{T_1,T_2}$ and

$$\mathbf{v}' = \mathsf{H}_2(\{K'_{i,j}\}_{i=1,j=1}^{T_1,T_2} \| m) = \mathbf{v},$$

confirming the correctness of the construction.

Our proposed scheme **CSIIBS+g** satisfies Properties 3.1, 3.2, and 3.3. Specifically, for an arbitrary epoch e, the signature generated using the user secret key usk_e in KUSS is indistinguishable from the signature generated using the user secret key usk in IBS. This holds because

$$\mathsf{usk}_e = (\mathbf{u}, \mathbf{x}^{(e)}) = (\mathbf{u}, \{x_{i,j}^{(e)}\}_{i=1,j=1}^{T_1,S_1}) = (\mathbf{u}, \{r_{i,j} - s_{u_i} - \Delta_{u_i,e}\}_{i=1,j=1}^{T_1,S_1}),$$

whereas the user secret key in IBS is defined as $\mathsf{usk} := (\mathbf{u}, \mathbf{x}) = (\mathbf{u}, \{r_{i,j} - s_{u_i}\}_{i=1,j=1}^{T_1,S_1})$. Since $\mathbf{x}$ and $\mathbf{x}^{(e)}$ are randomized, they are indistinguishable. Consequently, the signing algorithm $\mathsf{Sig}(\mathsf{usk}_e, \cdot)$ in KUSS operates as though it were using $\mathsf{Sig}(\mathsf{usk}, \cdot)$ in IBS, producing a signature of the form

$$\sigma = (\mathbf{v}, \mathbf{z}, \mathbf{X}) = (\{v_{i,j}\}_{i=1,j=1}^{T_1,T_2}, \{z_{i,j}\}_{i=1,j=1}^{T_1,T_2}, \{X_{i,j}\}_{i=1,j=1}^{T_1,S_1}).$$

It follows that the construction satisfies Property 3.2. Finally, to verify the signature we just need $\mathsf{mpk}_e = \{E_i^{(e)}\}_{i=1}^{S_0}$ to compute $(*)$ thereby satisfying Property 3.3.

5 Security Proof

In this section, we analyze the security of the constructions introduced in Section 4. In particular, we provide detailed proofs for $\mathsf{UF\text{-}KUSS\text{-}CMA}$ security for one pre- and one post-quantum scheme (i.e **BLMQ+g** and **CSIIBS+g**). We also outline proof strategies for establishing forward security and post-compromise security, illustrating the resilience of these constructions against the corresponding adversarial models.

5.1 Token Security

Theorem 5.1. *Suppose that the **BLMQ** scheme is unforgeable under adaptively chosen-message-and-identity attacks (UF-IBS-CMA), then the **BLMQ+g** scheme is unforgeable under adaptively chosen-message-identity-and-epoch attacks (UF-KUSS-CMA).*

Proof. Assume there exists an adversary $\mathcal{A}$ that succeeds in the experiment $\mathsf{Exp}^{\mathsf{UF\text{-}KUSS\text{-}CMA}}_{\mathbf{BLMQ+g},\mathcal{A}}(\lambda)$ with non-negligible probability ε and within time t. Let $\mathcal{B}$ be an adversary participating in the experiment $\mathsf{Exp}^{\mathsf{UF\text{-}IBS\text{-}CMA}}_{\mathbf{BLMQ},\mathcal{B}}(\lambda)$. On input λ, the challenger computes $(\mathsf{pp}, \mathsf{msk}, \mathsf{mpk})$ by invoking the Setup algorithm of **BLMQ** scheme and forwards $(\mathsf{pp}, \mathsf{mpk})$ to $\mathcal{B}$. Then, $\mathcal{B}$ forwards $(\mathsf{pp}, \mathsf{mpk}_0, \Delta_0)$ to $\mathcal{A}$, where $\Delta_0 \xleftarrow{\$} \mathbb{Z}_p^*$ initializes the epoch accumulator. On input $(\mathsf{pp}, \mathsf{mpk}_0, \Delta_0)$, $\mathcal{A}$ produces a forgery tuple $(\mathsf{id}^*, m^*, \sigma^*, e^*)$, where $\sigma^* = (h^*, S^*_{e^*})$ satisfying the following:

$$\tilde{r} = e(S^*_{e^*}, \mathsf{H}_1(\mathsf{id}^*)Q_{e^*} + \mathsf{mpk}_{e^*}) \cdot q^{-h^*},$$
$$h^* = \mathsf{H}_2(m^*, \tilde{r}).$$

Upon receving the forgery from $\mathcal{A}$, the adversary $\mathcal{B}$ outputs a forgery $(\mathsf{id}^*, m^*, \tilde{\sigma})$, where $\tilde{\sigma} = (h^*, \tilde{S})$ and $\tilde{S} = \Delta_{e^*} S^*_{e^*}$. The signature $\tilde{\sigma}$ is a valid forgery for the message m^* associated with id^* under the mpk of the **BLMQ** scheme as $\tilde{r}$ can still be recovered as follows:

$$\begin{aligned}\tilde{r} &= e(\tilde{S}, \mathsf{H}_1(\mathsf{id}^*)Q + \mathsf{mpk}) \cdot q^{-h^*} \\ &= q^{(x^* + h^* - h^*)} = q^{x^*}\end{aligned}$$

Thus, $\tilde{\sigma}$ constitutes a valid forged signature produced by $\mathcal{B}$ within time $t' \leq t + t_m$ with non-negligible probability ε, where t_m is the time required to compute Δ_{e^*} operations in $\mathbb{G}$. This result contradicts the UF-IBS-CMA security of the **BLMQ** scheme. □

Theorem 5.2. *If the **CSIIBS**, as introduced in [17], is unforgeable under adaptively chosen-message-and-identity attacks (UF-IBS-CMA), then the **CSIIBS+g** scheme, introduced in Section 4.2, is unforgeable under adaptively chosen-message-identity-and-epoch attacks (UF-KUSS-CMA), according to Definition 3.3.*

Proof. Assume there exists an efficient adversary $\mathcal{A}$ that breaks the UF-KUSS-CMA security of **CSIIBS+g** (Definition 3.3) with non-negligible probability. We show that this implies the existence of a forger $\mathcal{B}$ that breaks the UF-IBS-CMA security of **CSIIBS**. To do so, $\mathcal{B}$ leverages $\mathcal{A}$'s success in the experiment $\mathsf{Exp}^{\mathsf{UF\text{-}KUSS\text{-}CMA}}_{\mathbf{CSIIBS+g},\mathcal{A}}(\lambda)$ to participate in $\mathsf{Exp}^{\mathsf{UF\text{-}IBS\text{-}CMA}}_{\mathbf{CSIIBS},\mathcal{B}}(\lambda)$ (Definition 2.3). Specifically, $\mathcal{B}$ acts as the challenger for $\mathcal{A}$, simulating all oracles and responses using its own oracle access.

Setup. In the $\mathsf{Exp}^{\mathsf{UF\text{-}IBS\text{-}CMA}}_{\mathbf{CSIIBS},\mathcal{B}}(\lambda)$ experiment, the challenger computes $(\mathsf{pp}$, msk_0 , $\mathsf{mpk}_0)$ by invoking the Setup algorithm of **CSIIBS**, and forwards $(\mathsf{pp}, \mathsf{mpk}_0)$ to

$\mathcal{B}$. $\mathcal{B}$ forwards $(\mathsf{pp}, \mathsf{mpk}_0, \boldsymbol{\Delta}_0)$ to $\mathcal{A}$, where $\boldsymbol{\Delta}_0 := \mathbf{0}$ initializes the epoch accumulator. It also initializes two empty lists: $\mathsf{idList}, \mathsf{mList}$, and an epoch sequence epSeq, with $\mathsf{epSeq} := (\boldsymbol{\Delta}_0) = (\boldsymbol{\Delta}_i)_{i=0}^{0}$.

Simulation of Queries. $\mathcal{A}$ can issue polynomially many adaptive queries to the following oracles: $\mathcal{O}_{\mathsf{Ext}}, \mathcal{O}_{\mathsf{N}}, \mathcal{O}_{\mathsf{U}}$, and $\mathcal{O}_{\mathsf{Sign}}$. $\mathcal{B}$ simulates these oracles using its own access to the $\mathsf{Exp}^{\mathsf{UF\text{-}IBS\text{-}CMA}}_{\mathsf{CSIIBS},\mathcal{B}}(\lambda)$ experiment:

- *Simulating* $\mathcal{O}_{\mathsf{Ext}}(\mathsf{msk}_e, \cdot)$: For a queried identity id, if $\mathsf{id} \in \mathsf{idList}$, return $\perp$. Otherwise, $\mathcal{B}$ queries its extracting oracle with id to obtain a secret key usk. For epoch e, it computes the epoch-specific user secret key $\mathsf{usk}_e := (\mathbf{u}, \mathbf{x}^{(e)} = \{x_{i,j} - \Delta_{u_i,e}\}_{i=1,j=1}^{T_1,S_1})$, and updates $\mathsf{idList} \leftarrow \mathsf{idList} \cup \{\mathsf{id}\}$.
- *Simulating* $\mathcal{O}_{\mathsf{N}}(\lambda, \cdot)$: Upon receiving a query to this oracle at epoch e, $\mathcal{B}$ samples a new random update token $\boldsymbol{\delta}_{e+1}$ for epoch $e+1$.
- *Simulating* $\mathcal{O}_{\mathsf{U}}(\cdot, \cdot)$: Upon receiving a query to this oracle at epoch e on input $\boldsymbol{\delta}_{e+1}$ and $k_e = (\mathsf{msk}_e, \mathsf{mpk}_e, \mathsf{usk}_e, \boldsymbol{\Delta}_e)$, $\mathcal{B}$ computes $\boldsymbol{\Delta}_{e+1} = \boldsymbol{\delta}_{e+1} + \boldsymbol{\Delta}_e$, $\mathsf{mpk}_{e+1} = \{[s_i + \Delta_{i,e} + \delta_{i,e+1}]E_0\}_{i=1}^{S_0}$, and $\mathsf{usk}_{e+1} = (\mathbf{u}, \mathbf{x}^{(e+1)}) = (\mathbf{u}, \{r_{i,j} - s_{u_i} - \Delta_{u_i,e} - \delta_{u_i,e+1}\}_{i=1,j=1}^{T_1,S_1})$ for all $\mathsf{id} \in \mathsf{idList}$. Finally, $\mathcal{B}$ updates the sequence $\mathsf{epSeq} \leftarrow (\boldsymbol{\Delta}_i)_{i=0}^{e+1}$.
- *Simulating* $\mathcal{O}_{\mathsf{Sign}}(\mathsf{usk_e}, \cdot)$: Upon receiving a query to this oracle on input a message $m \in \mathcal{M}$ at epoch e, $\mathcal{B}$ produces a signature $\sigma = (\mathbf{v}, \mathbf{z}, \mathbf{X})$ on m under mpk using its oracle. $\mathcal{B}$, then modifies $\mathbf{z}$ by setting $z_{i,j} \leftarrow z_{i,j} - \Delta_{u_i,e} \mod N$ for which $v_{i,j} = 0$. $\mathcal{B}$ return the modified signature $\sigma' = (\mathbf{v}, \mathbf{z}', \mathbf{X})$ to $\mathcal{A}$, and updates the list $\mathsf{mList} \leftarrow \mathsf{mList} \cup \{m\}$.

Extracting the forgery: $\mathcal{A}$ eventually submits a forgery $(\mathsf{id}^*, m^*, \sigma^*, e^*)$ to $\mathcal{B}$, where $\sigma^* = (\mathbf{v}^*, \mathbf{z}^*, \mathbf{X}^*)$. $\mathcal{B}$ modifies the signature σ^* to frame a forgery under the mpk by retrieving Δ_{e^*} from epSeq and setting $z^*_{i,j} \leftarrow z^*_{i,j} + \Delta_{u_i,e^*} \mod N$ for which $v_{i,j} = 0$. Thereby, $\mathcal{B}$ creates a forged signature $\bar{\sigma}^* = (\mathbf{v}^*, \bar{\mathbf{z}}^*, \mathbf{X}^*)$, and outputs $(\mathsf{id}^*, m^*, \bar{\sigma}^*)$ as a forgery to its challenger.

For the analysis, assume that the probability with which $\mathcal{A}$ wins in the experiment $\mathsf{Exp}^{\mathsf{UF\text{-}KUSS\text{-}CMA}}_{\mathsf{CSIIBS+g},\mathcal{A}}(\lambda)$ is non-negligible. We shall now demonstrate that $\mathcal{B}$ provides a perfect simulation of the oracle $\mathcal{O}_{\mathsf{Sign}}$ on message m. The signature $\sigma = (\mathbf{v}, \mathbf{z}, \mathbf{X})$ on m under $\mathsf{mpk} = \{E_i\}_{i=1}^{S_0}$ received by $\mathcal{B}$ from its own signing oracle consists of $\mathbf{v} = \{v_{i,j}\}_{i=1,j=1}^{T_1,T_2}$, $\mathbf{z} = \{z_{i,j}\}_{i=1,j=1}^{T_1,T_2} = \{k_{i,j} - x_{i,v_{i,j}}\}_{i=1,j=1}^{T_1,T_2}$, and $\mathbf{X} = \{X_{i,j}\}_{i=1,j=1}^{T_1,S_1} = \{[x_{i,j}]E_{u_i}\}_{i=1,j=1}^{T_1,S_1}$, where $\{v_{i,j}\}_{i=1,j=1}^{T_1,T_2} = \mathsf{H}_2(K_{i,j} \| m)$ with $K_{i,j} = [k_{i,j}]E_{u_i}$ for all $k_{i,j} \in \mathbb{Z}_N$, $i \in [T_1]$, and $j \in [T_2]$. By modifying $\mathbf{z}$ via $z'_{i,j} := z_{i,j} - \Delta_{u_i,e} \mod N$ for which $v_{i,j} = 0$, we obtain $\mathbf{z}' := \{z'_{i,j}\}_{i=1,j=1}^{T_1,T_2}$. We still retrieve $\mathbf{v} = \{v_{i,j}\}_{i=1,j=1}^{T_1,T_2}$ under mpk_e as follows:

- $[z'_{i,j}]E_{u_i}^{(e)} = [k_{i,j} - x_{i,v_{i,j}} - \Delta_{u_i,e}]E_{u_i}^{(e)} = [k_{i,j} - \Delta_{u_i,e}]E_{u_i}^{(e)} = [k_{i,j}]E_{u_i}$,
- $[z'_{i,j}]X_{i,v_{i,j}} = [k_{i,j} - x_{i,v_{i,j}}][x_{i,v_{i,j}}]E_{u_i} = [k_{i,j}]E_{u_i}$,

in case $v_{i,j} = 0$, and $v_{i,j} \neq 0$, respectively. Thus, $\sigma = (\mathbf{v}, \mathbf{z}', \mathbf{X})$ is a signature on m under the mpk_e associated to identity id.

Similarly, the forgery of $\mathcal{B}$ is computed from the forgery of $\mathcal{A}$. Let $\mathcal{A}$ submit a valid forgery $(\mathsf{id}^*, m^*, \sigma^*, e^*)$, where the signature is $\sigma^* = (\{v^*_{i,j}\}^{T_1,T_2}_{i=1,j=1}, \{z^*_{i,j}\}^{T_1,T_2}_{i=1,j=1}, \{X^*_{i,j}\}^{T_1,S_1}_{i=1,j=1})$. Then, $\mathcal{B}$ modifies the signature σ^* to frame a forgery under the mpk by setting $\bar{z}^*_{i,j} := z^*_{i,j} + \Delta_{u_i,e^*} \mod N$ for which $v^*_{i,j} = 0$, thereby under mpk the hash value $\mathbf{v}^* = \{v^*_{i,j}\}^{T_1,T_2}_{i=1,j=1}$ can be still retrieved as follows:

- $[\bar{z}^*_{i,j}]E_{u_i} = [k^*_{i,j} - x^*_{i,v^*_{i,j}} + \Delta_{u_i,e^*}]E_{u_i} = [k^*_{i,j} + \Delta_{u_i,e^*}]E_{u_i} = \left[k^*_{i,j}\right] E^{(e^*)}_{u_i}$.
- $[\bar{z}^*_{i,j}]X^*_{i,v^*_{i,j}} = [k^*_{i,j} - x^*_{i,v^*_{i,j}}][x^*_{i,v^*_{i,j}}]E^{(e^*)}_{u_i} = [k^*_{i,j}]E^{(e^*)}_{u_i}$.

in case $v^*_{i,j} = 0$, and $v^*_{i,j} \neq 0$, respectively. Thus, $\bar{\sigma}^* = (\mathbf{v}^*, \bar{\mathbf{z}}^*, \mathbf{X}^*)$ is a valid signature on m^* under the mpk for identity id^*. Moreover, $\mathcal{B}$ queries identical messages as $\mathcal{A}$ while responding to signing queries for $\mathcal{A}$, and therefore whenever $\mathcal{A}$ wins in the experiment $\mathsf{Exp}^{\mathsf{UF\text{-}KUSS\text{-}CMA}}_{\mathsf{CSIIBS+g},\mathcal{A}}(\lambda)$, then $\mathcal{B}$ wins in the experiment $\mathsf{Exp}^{\mathsf{UF\text{-}IBS\text{-}CMA}}_{\mathsf{CSIIBS},\mathcal{B}}(\lambda)$. □

5.2 Forward Security

Forward security in KUSS is achieved if and only if the following conditions are met:

1. A client's entry triggers a transition to epoch $e + 1$ by invoking Next and Update.
2. Upon entry, the new client receives only Δ_{e+1} but not δ_{e+1}.

The second condition ensures forward security in cases of re-entry. Specifically, if a device lacks knowledge of previously used Δ, disclosing δ would pose no risk. However, if the device had been a group member two sessions before, knowledge of δ could enable the reconstruction of the previous Δ value. Consider the following scenario:

$$\begin{aligned} \Delta_0 &\xleftarrow{\$} \mathbb{Z}^*_p, \quad \delta_{1,2} \xleftarrow{\$} \mathbb{Z}^*_p, \\ \Delta_1 &= \Delta_0 \cdot \delta_1, \\ \Delta_2 &= \Delta_0 \cdot \delta_1 \cdot \delta_2, \end{aligned}$$

where Δ_0, Δ_1, and Δ_2 represent the shared secrets for epochs 0, 1, and 2, respectively.

Now, for schemes introduced in Section 4.1, assume a client who was a group member in epoch 0, but not in epoch 1, re-joins the group in epoch 2. This client knows Δ_0 and Δ_2, but lacks knowledge of Δ_1, δ_1, and δ_2. If this client learns δ_2, it could compute: $\frac{\Delta_2}{\Delta_0 \cdot \delta_2} = \delta_1$ and learn $\Delta_1 = \Delta_0 \cdot \delta_1$. However, with only Δ_0 and Δ_2, reconstructing Δ_1 would require: $\frac{\Delta_2}{\delta_2} = \Delta_1$ and $\Delta_0 \cdot \delta_1 = \Delta_1$. According to Lemma 2.2, the probability of obtaining such δ_1 or δ_2 is $\epsilon = \frac{1}{p-1}$. Thus, by updating Δ upon a client's entry and refraining from disclosing the current δ to new (or re-entering) members, forward security is preserved in KUSS.

In the case of **CSIIBS+g**, introduced in Section 4.2, the member is still unable to compute $\boldsymbol{\Delta}_{e-1}$ since the probability of finding such $\boldsymbol{\delta}_{e-1}$ or $\boldsymbol{\delta}_e$ tokens such that

$$\begin{aligned}\boldsymbol{\Delta}_{e-1} &= \boldsymbol{\Delta}_{e-2} + \boldsymbol{\delta}_{e-1} \mod N \\ &= \boldsymbol{\Delta}_e - \boldsymbol{\delta}_e \mod N\end{aligned}$$

is computationally infeasible: $\frac{1}{h(\mathfrak{D})} = \frac{1}{N} \approx \frac{1}{p^{1/2}} \approx \frac{1}{2^\lambda}$.

5.3 Post-Compromise Security

Theorem 5.3 (KUSS Post-Compromise Security). *Assuming the hardness of the One-More Discrete Logarithm Problem (1MDLP) [35], it is computationally infeasible for an adversary to compute δ_{e+1} from mpk_{e+1} in the $\mathbf{X}+\mathbf{g}$ construction, where $\mathbf{X} \in \{\mathbf{Hess}, \mathbf{BLMQ}, \mathbf{GG}, \mathbf{vBNN}\}$.*

Proof. Let $\mathcal{A}$ have access to the Oracle $\mathcal{O}_{\mathsf{N}}$, which transitions the global state at arbitrary epoch e to epoch $e+1$, and to $\mathcal{O}_{\mathsf{U}}(\cdot, \mathsf{mpk}_e, \cdot, \cdot)$, which returns a $\mathsf{mpk}_{e+1} = \delta_{e+1}\mathsf{mpk}_e$ (where $\delta_{e+1} \xleftarrow{\$} \mathbb{Z}_p^*$). In considered schemes introduced in Section 4.1, $\mathsf{mpk} = \mathsf{msk}\,P$, where $\mathsf{msk} \xleftarrow{\$} \mathbb{Z}_p^*$ and P is the subgroup generator of an elliptic curve. Thus, we have:

$$\begin{aligned}\mathsf{mpk}_{e+1} &= (\delta_{e+1} \cdot \Delta_e \cdot \mathsf{msk}_0)P \\ &= (\delta_{e+1} \cdot \mathsf{msk}_e)P \\ &= \mathsf{msk}_{e+1}\,P\end{aligned}$$

According to Lemma 2.2, given msk_e the probability $P(k \xleftarrow{\$} \mathbb{Z}_p^* | \mathsf{msk}_{e+1} = k \cdot \mathsf{msk}_e) = \frac{1}{p-1}$. The output of $\mathcal{O}_{\mathsf{U}}(\cdot, \mathsf{mpk}_e, \cdot, \cdot)$ fits the definition of *random target points* in the `Chall` Oracle in [31], where $Y \xleftarrow{\$} \mathbb{G}$. Hence, the advantage for $\mathcal{A}$ to compute δ_{e+1} from mpk_{e+1} is bounded by the 1MDLP [35]. □

In a similar argument, in the case of **CSIIBS+g**, all signatures and keys from epochs following the compromise remain secure. Let the adversary $\mathcal{A}$ have access to the oracle $\mathcal{O}_{\mathsf{N}}$, which advances the global state from epoch e to epoch $e+1$, and to $\mathcal{O}_{\mathsf{U}}(\cdot, \mathsf{mpk}_e, \cdot, \cdot)$, which returns $\mathsf{mpk}_{e+1} = \langle [\boldsymbol{\delta}_{e+1}], \mathsf{mpk}_e \rangle$, where $\boldsymbol{\delta}_{e+1} \xleftarrow{\$} (\mathbb{Z}/N\mathbb{Z})^{S_0}$. Then, we have

$$\begin{aligned}\mathsf{mpk}_{e+1} &= \{E_i^{(e+1)}\}_{i=1}^{S_0} \\ &= \{[s_i + \Delta_{i,e} + \delta_{i,e+1}]E_0\}_{i=1}^{S_0} \\ &= \{[\delta_{i,e+1}]E_i^{(e)}\}_{i=1}^{S_0} \\ &= \langle [\boldsymbol{\delta}_{e+1}], \mathsf{mpk}_e \rangle.\end{aligned}$$

However, Problem 2.1 implies that finding such a token $\boldsymbol{\delta}_{e+1} = \{\delta_{i,e+1}\}_{i=1}^{S_0}$ satisfying $\mathsf{mpk}_{e+1} = \langle [\boldsymbol{\delta}_{e+1}], \mathsf{mpk}_e \rangle$ is computationally infeasible.

5.4 Known weaknesses in BLMQ+g

Let a client be expelled at epoch $e-1$ and rejoin at epoch $e+1$. In **BLMQ+g**, a signature from epoch e has the form $\sigma_e = (h, S_e)$, where $S_e = (x+h)\,\mathsf{usk}_e$, or equivalently, $S_e = (x+h)\cdot \Delta_e^{-1}\,\mathsf{usk}$, with h being the hash of the message and $x \xleftarrow{\$} \mathbb{Z}_p^*$. Since the client does not know Δ_e, the signature appears perfectly random. However, the client knows $S_{e-1} = (x+h)\cdot\Delta_{e-1}^{-1}\,\mathsf{usk}$ and $S_{e+1} = (x+h)\cdot \Delta_{e+1}^{-1}\,\mathsf{usk}$. If, and only if, the client knows both Δ_{e+1} and Δ_{e-1}, she can compute: $S^*_{e-1} := \Delta_{e-1}\,S_{e-1}$, and $S^*_{e+1} := \Delta_{e+1}\,S_{e+1}$. If $S^*_{e-1} = S^*_{e+1}$, she can deduce that the signature was generated with the same original usk (even without knowing usk).

In practice, this vulnerability allows a replay attack using messages signed with previously used Δ's from other users in the system. Let an attacker know the shared secret Δ_e from a recorded signature σ_e at some epoch $e < e^*$, and have access to the current Δ_{e^*}. The attacker can calculate a valid signature $\sigma_{e^*} := (h, (\Delta_e \cdot \Delta_{e^*}^{-1})S_e)$. Although the attacker cannot change the message or identity, it can evolve the signature to a new epoch. However, such a replay attack can be easily mitigated with state-of-the-art mechanisms, such as sequence numbers or timestamps, included in the content of every message m. It is worth noting that such a replay attack is possible in all the original schemes. Thus, the KUSS transformation mathematically prevents this attack for all schemes except **BLMQ**.

6 Conclusion

Reliable key updates for asymmetric keys are critical in the event of key compromise or member revocation within a communication group. Efficient key updates are particularly important for sender authentication in groups comprising resource-constrained clients. Existing mechanisms, such as X.509 certificates or traditional IBS schemes, address revocation but often incur substantial computational, networking, or management overhead. Efficient symmetric re-keying mechanisms, in contrast, offer practical solutions.

In this work, we introduced a shared secret to facilitate efficient key updates in IBS schemes. Leveraging symmetric re-keying within group infrastructures, our protocol, gIBS, extends classical IBS schemes to group settings, providing provable verification of authentication keys and ensuring both forward and backward secrecy of the shared update token. Our KUSS framework provides a formal foundation for analyzing key secrecy, independence, and existential unforgeability under adaptive attacks. The proposed transformation, applied to pairing-based and isogeny-based schemes, maintains correctness while incurring minimal computational overhead and reducing network load, especially when combined with efficient key distribution mechanisms like LKH. This work demonstrates the generality of the approach and its applicability beyond IoT, including post-quantum settings. Future work includes extending security proofs to Schnorr-like schemes, validating compatibility with other elliptic curve IBS constructions, and exploring applications to alternative cryptographic settings, such as lattices.

References

1. Whyte, W., Weimerskirch, A., Kumar, V., Hehn, T.: A Security Credential Management System for V2V Communications. In: 2013 IEEE Vehicular Networking Conference, pp. 1–8. IEEE (2013)
2. Boneh, D., Franklin, M.: Identity-Based Encryption from the Weil Pairing. In: Annual International Cryptology Conference, pp. 213–229. Springer (2001)
3. Boldyreva, A., Goyal, V., Kumar, V.: Identity-Based Encryption with Efficient Revocation. In: Proceedings of the 15th ACM Conference on Computer and Communications Security, pp. 417–426. ACM (2008)
4. Chen, J., Lim, H.W., Ling, S., Wang, H., Nguyen, K.: Revocable Identity-Based Encryption from Lattices. In: Australasian Conference on Information Security and Privacy, pp. 390–403. Springer (2012)
5. Gentry, C., Silverberg, A.: Hierarchical ID-Based Cryptography. In: International Conference on the Theory and Application of Cryptology and Information Security, pp. 548–566. Springer (2002)
6. Shamir, A.: Identity-Based Cryptosystems and Signature Schemes. In: Workshop on the Theory and Application of Cryptographic Techniques, pp. 47–53. Springer (1984)
7. Au, M.H., Liu, J.K., Yuen, T.H., Wong, D.S.: Practical Hierarchical Identity Based Encryption and Signature Schemes Without Random Oracles. Cryptology ePrint Archive (2006)
8. Seo, J.H., Emura, K.: Revocable Identity-Based Encryption Revisited: Security Model and Construction. In: International Workshop on Public Key Cryptography, pp. 216–234. Springer (2013)
9. Dodis, Y., Katz, J., Xu, S., Yung, M.: Strong Key-Insulated Signature Schemes. In: International Workshop on Public Key Cryptography, pp. 130–144. Springer (2002)
10. Lehmann, A., Tackmann, B.: Updatable Encryption with Post-Compromise Security. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 685–716. Springer (2018)
11. Galindo, D., Garcia, F.D.: A Schnorr-Like Lightweight Identity-Based Signature Scheme. In: International Conference on Cryptology in Africa, pp. 135–148. Springer (2009)
12. Schnorr, C.-P.: Efficient Signature Generation by Smart Cards. Journal of Cryptology **4**(3), 161–174 (1991)
13. Cao, X., Kou, W., Dang, L., Zhao, B.: IMBAS: Identity-based multi-user broadcast authentication in wireless sensor networks. Computer Communications **31**(4), 659–667 (2008)
14. Hess, F.: Efficient Identity Based Signature Schemes Based on Pairings. In: International Workshop on Selected Areas in Cryptography, pp. 310–324. Springer (2002)
15. Barreto, P.S.L.M., Libert, B., McCullagh, N., Quisquater, J.-J.: Efficient and Provably-Secure Identity-Based Signatures and Signcryption from Bilinear Maps. In: International Conference on the Theory and Application of Cryptology and Information Security, pp. 515–532. Springer (2005)
16. Beullens, W., Kleinjung, T., Vercauteren, F.: CSI-FiSh: Efficient Isogeny Based Signatures Through Class Group Computations. In: International Conference on the Theory and Application of Cryptology and Information Security, pp. 227–247. Springer (2019)

17. Shaw, S., Dutta, R.: Identification Scheme and Forward-Secure Signature in Identity-Based Setting from Isogenies. In: International Conference on Provable Security, pp. 309–326. Springer (2021)
18. Ohtake, G., Hanaoka, G., Ogawa, K.: An Efficient Strong Key-Insulated Signature Scheme and Its Application. In: European Public Key Infrastructure Workshop, pp. 150–165. Springer (2008)
19. Palombini, F., Tiloca, M.: RFC 9594: Key Provisioning for Group Communication Using Authentication and Authorization for Constrained Environments (ACE). RFC Editor (2024)
20. Tiloca, M., Park, J., Palombini, F.: Key Management for OSCORE Groups in ACE. Internet-Draft draft-ietf-ace-key-groupcomm-oscore-02, Work in Progress (2019)
21. Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R., Polk, W.: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280 (2008)
22. Malpani, A., Galperin, S., Adams, C.: X.509 Internet Public Key Infrastructure Online Certificate Status Protocol-OCSP. RFC 6960 (2013)
23. Libert, B., Peters, T., Yung, M.: Scalable Group Signatures with Revocation. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 609–627. Springer (2012)
24. Zhou, S., Lin, D.: Shorter Verifier-Local Revocation Group Signatures from Bilinear Maps. In: International Conference on Cryptology and Network Security, pp. 126–143. Springer (2006)
25. Felde, N.G., Grundner-Culemann, S., Guggemos, T.: Authentication in Dynamic Groups Using Identity-Based Signatures. In: 2018 14th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), pp. 1–6. IEEE (2018)
26. Liu, Z., Lai, Y., Ren, X., Bu, S.: An Efficient LKH Tree Balancing Algorithm for Group Key Management. In: 2012 International Conference on Control Engineering and Communication Technology, pp. 1003–1005. IEEE (2012)
27. Silverman, J.H.: The Arithmetic of Elliptic Curves, vol. 106. Springer (2009)
28. Baugher, M., Canetti, R., Dondeti, L., Lindholm, F.: Multicast Security (MSEC) Group Key Management Architecture. RFC 4046 (2005)
29. Challal, Y., Seba, H.: Group Key Management Protocols: A Novel Taxonomy. International Journal of Information Technology **2**(1), 105–118 (2005)
30. Smyslov, V., Weis, B.: Group Key Management using IKEv2. Internet-Draft draft-ietf-ipsecme-g-ikev2-23, Work in Progress (2025)
31. Bellare, M., Namprempre, C., Neven, G.: Security Proofs for Identity-Based Identification and Signature Schemes. Journal of Cryptology **22**(1), 1–61 (2009)
32. Bellare, M., Yee, B.: Forward-Security in Private-Key Cryptography. In: Cryptographers' Track at the RSA Conference, pp. 1–18. Springer (2003)
33. Itkis, G.: Forward Security, Adaptive Cryptography: Time Evolution. In: Handbook of Information Security, vol. 3, pp. 927–944. John Wiley and Sons (2006)
34. Kim, Y., Perrig, A., Tsudik, G.: Simple and Fault-Tolerant Key Agreement for Dynamic Collaborative Groups. In: Proceedings of the 7th ACM Conference on Computer and Communications Security, pp. 235–244. ACM (2000)
35. Koblitz, N., Menezes, A.: Another Look at Non-Standard Discrete Log and Diffie-Hellman Problems. Journal of Mathematical Cryptology **2**(4), 311–326 (2008)